

# Reviewer Pack — Minimal Geometric Fluctuation of Tropical Curves vs Monotone...

Entry 2c2530d10446 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 12:37:03 UTC

## Minimal Geometric Fluctuation of Tropical Curves vs Monotone Circuit Lower Bounds for k-CLIQUE

Entry ID: 2c2530d10446 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 12:37:03 UTC

### 1. Conjecture

**Field A** (mathematical branch): Tropical Geometry **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity for k-CLIQUE

#### Statement:

[‘For every  $n$ -vertex graph  $G$ , the minimal geometric fluctuation of its tropical curve over a suitable semiring is at least  $\Omega(n^{\{1/2\}})$  for any monotone circuit computing k-CLIQUE on  $G$  with size  $n^{\Omega(k)\{1/4\}}$ .’, ‘Furthermore, there exists an effective mapping from graph instances to tropical curves such that the geometric fluctuation of the tropical curve serves as a lower bound on the monotone circuit complexity.’, ‘This conjecture is falsified if there exists a graph  $G$  for which the minimal geometric fluctuation of its tropical curve is  $o(n^{\{1/2\}})$  and a monotone circuit computing k-CLIQUE on  $G$  with size  $n^{\Omega(k)\{1/4\}}$ .’]

#### Rationale (proposer’s reasoning):

[‘Tropical geometry provides a combinatorial framework that can potentially reveal hidden complexities in the computation of k-CLIQUE.’, ‘The geometric fluctuation measure captures the complexity of the interaction between vertices in a graph and could serve as an invariant for monotone circuits.’, ‘If this conjecture is true, it would suggest a new method for proving lower bounds on monotone circuit complexity.’]

**Taxonomy category:** MONOTONE\_CLIQUE (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** e8de5fa23292d406

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The conjecture is supported if the average geometric fluctuation of tropical curves across all  $n \leq 40$  vertices graphs  $G$  is at least  $\Omega(n^{\{1/2\}})$  for monotone circuits computing k-CLIQUE with size  $n^{\Omega(k)\{1/4\}}$ , and falsified if any graph  $G$  has a geometric fluctuation less than  $o(n^{\{1/2\}})$ .

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict   | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|-----------|------------|------------------|------------------|
| RELATIVIZATION | UNCERTAIN | 1.00       | HITS             | UNCERTAIN        |
| NATURAL_PROOFS | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |

### 4. Novelty audit

**Verdict:** ADJACENT\_OK against 6 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - tropical geometry AND monotone circuit complexity FOR k-CLIQUE - geometric fluctuation IN tropical curves AND lower bound ON monotone circuit complexity - mapping FROM graph instances TO tropical curves AND geometric fluctuation AS lower bound

**Top relevant hits considered:** - [http://arxiv.org/abs/1206.1925v1] Counting Algebraic Curves with Tropical Geometry - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/2309.17302v2] Geometry of tropical extensions of hyperfields - [http://arxiv.org/abs/math/0511713v2] Tropical Plane Geometric Constructions: a Transfer Technique in Tropical Geometry - [http://arxiv.org/abs/1306.3497v2] The number of vertices of a tropical curve is bounded by its area - [http://arxiv.org/abs/1204.3875v2] Tropicalizing vs Compactifying the Torelli morphism

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

#### 5.1 Generated Python source

```
import random
import math
from itertools import combinations

def generate_random_graph(n):
    edges = set()
    for u in range(n):
        for v in range(u + 1, n):
            if random.random() < 0.5:
                edges.add((u, v))
    return edges

def max_plus_tropical_curve(edges):
    n = len(edges) + 1
    curve = [[float('-inf')] * n for _ in range(n)]
    for u, v in edges:
        curve[u][v] = 1
        curve[v][u] = 1
    for k in range(n):
        for i in range(n):
            for j in range(n):
                curve[i][j] = max(curve[i][j], curve[i][k] + curve[k][j])
```

```

    return curve

def geometric_fluctuation(curve):
    n = len(curve)
    total = 0
    count = 0
    for i in range(n):
        for j in range(i + 1, n):
            if curve[i][j] != float('-inf'):
                total += abs(curve[i][j])
                count += 1
    if count == 0:
        return 0
    return total / count

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        instances_tested = 0
        total_fluctuation = 0
        for _ in range(5): # Ensure at least 5 instances per size
            edges = generate_random_graph(n)
            curve = max_plus_tropical_curve(edges)
            fluctuation = geometric_fluctuation(curve)
            results.append(fluctuation)
            total_fluctuation += fluctuation
            instances_tested += 1
        mean_fluctuation = total_fluctuation / instances_tested
        conjecture_holds = mean_fluctuation >= math.sqrt(n)
        counterexample = "" if conjecture_holds else f"n={n}, avg_fluctuation={mean_fluctuation}"
        results.append({
            "metric_name": "geometric_fluctuation",
            "metric_value": mean_fluctuation,
            "instances_tested": instances_tested,
            "conjecture_holds": conjecture_holds,
            "counterexample": counterexample
        })
    return {
        "seed": seed,
        **results[-1]
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_fluctuation = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_fluctuation} std=0 support_fraction=1")

```

```

elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_fluctuation} std=0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='n={results[0]['instances_tested']}', avg_fluctuation={avg_fluctuation}")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9ec8e163.py", line 83, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9ec8e163.py", line 58, in run_trial
    curve = max_plus_tropical_curve(edges)
            ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9ec8e163.py", line 29, in max_plus_tropical_curve
    curve[u][v] = 1
    ~~~~~^
IndexError: list assignment index out of range

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test crashed before producing data, which means that it did not complete successfully to provide evidence for or against the conjecture. | next: Re-run the test with appropriate error handling and debugging to ensure it completes without crashing.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | glm4:latest      | 0         | 0   | 15667        |
| 2 | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 9657         |
| 3 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 8768         |
| 4 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 9400         |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15107        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 8053         |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 8135         |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 9944         |

| # | Phase | Provider      | Model       | Tokens in | out | Latency (ms) |
|---|-------|---------------|-------------|-----------|-----|--------------|
| 9 | judge | ollama_remote | glm4:latest | 0         | 0   | 11653        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 96384 ms total latency. Provider mix: {'ollama\_remote': 9}

*(full prompt+response transcripts available in research/audit/2c2530d10446.jsonl)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/2c2530d10446.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/2c2530d10446.tar.gz (if generated)